IT457
Cloud Computing
Synchronization In Distributed systems

Week 8-Lec2

Clock Synchronization

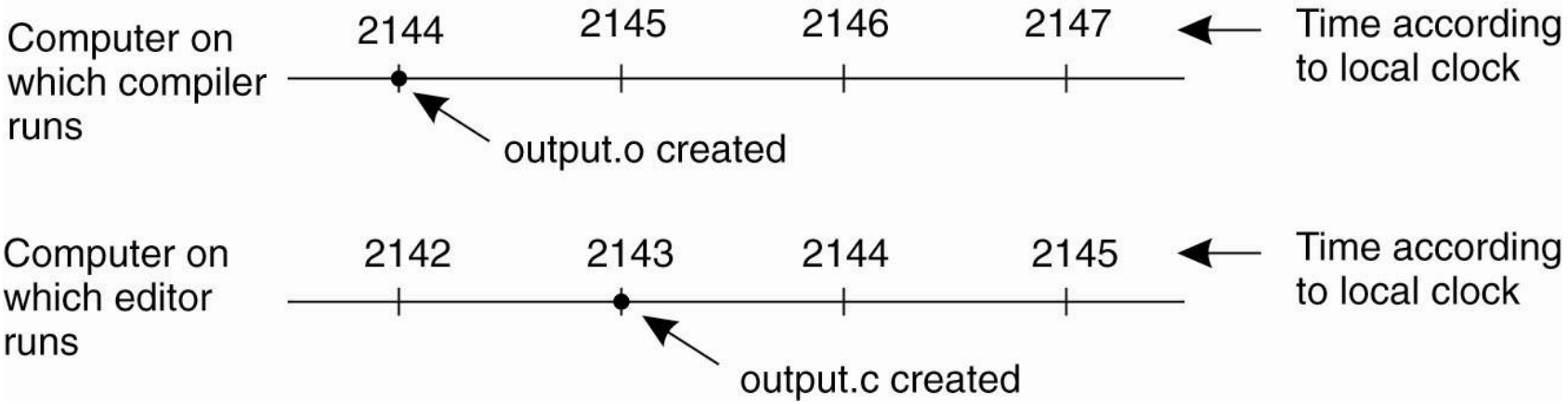


Figure 6-1. When each machine has its own clock, an event that occurred after another event may nevertheless be assigned an earlier time.

Global Time & Global State of Distributed Systems

Asynchronous distributed systems consist of several *processes* without common memory which communicate (solely) via *messages* with unpredictable transmission delays

Global time & global state are hard to realize in distributed systems

- Processes are distributed geographically
- Rate of event occurrence can be high (unpredictable)
- Event execution times can be small

We can only *approximate* the global view

- *Simulate synchronous* distributed system on given asynchronous systems
- *Simulate* a *global time* – Logical Clocks
- *Simulate* a *global state* – Global Snapshots

Simulating global time

- Clocks in a distributed system drift
  - Relative to each other
  - Relative to a real world clock
  - Clock Skew versus Drift
    - Clock Skew = Relative Difference in clock values of two processes
    - Clock Drift = Relative Difference in clock frequencies (rates) of two processes
- Clock synchronization is needed to simulate global time

– Correctness – consistency, fairness
– deviation between two clocks on any two machines within a specified bound, known as the precision
– accuracy, we aim to keep the clock bound to a value α

► Internal synchronization: keep clocks precise

► External synchronization: keep clocks accurate

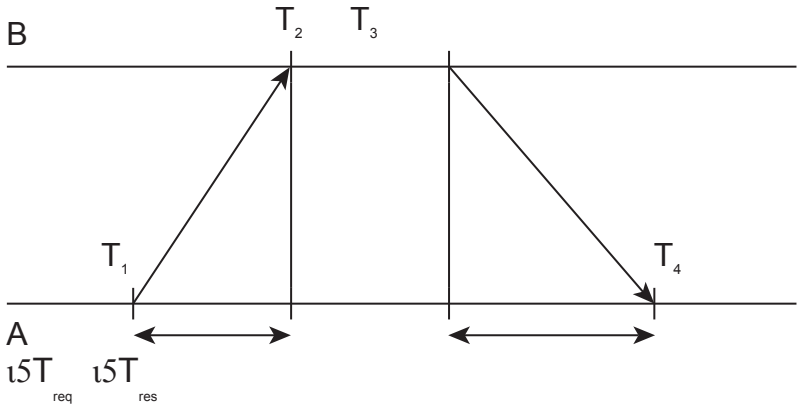
Client sends request (T1) : 10:00:00.000
Server receives it (T2): 10:00:00.080
Server sends response (T3): 10:00:00.081
Client receives response(T4): 10:00:00.161

RTT = 161ms
One way delay = 80.5ms

Server time is 10:00:00.081
Server timestamp = 10:00:00.161 - 80.5

Detecting and adjusting incorrect times

Getting the current time from a time server



Computing the relative offset θ and delay δ

Assumption: $\delta_{Treq} = T_2 - T_1 \approx T_4 - T_3 = \delta_{Tres}$

$$\theta = (T_2 - T_1) + (T_3 - T_4)/2$$

$$\delta = (T_4 - T_1) - (T_3 - T_2)/2$$

The Happened-before relationship

Issue

What usually matters is not that all processes agree on exactly what time it is, but that they agree on the order in which events occur.

Requires a notion of ordering.

Lamport's Logical Clocks (1)

The "happens-before" relation $\rightarrow$ can be observed directly in two situations:

- If *a* and *b* are events in the same process, and *a* occurs before *b*, then $a \rightarrow b$ is true.
- If a is the event of a message being sent by one process, and *b* is the event of the message being received by another process, then $a \rightarrow b$

Lamport's Logical Clocks (2)

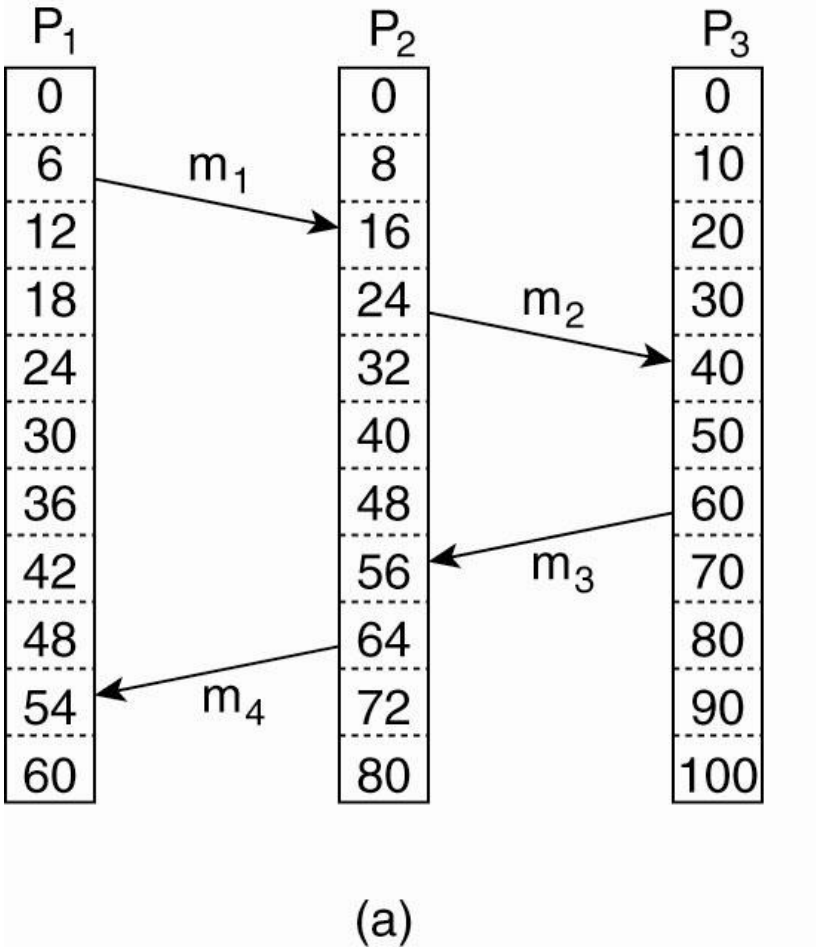


Figure 6-9. (a) Three processes, each with its own clock. The clocks run at different rates.

Lamport's Logical Clocks (3)

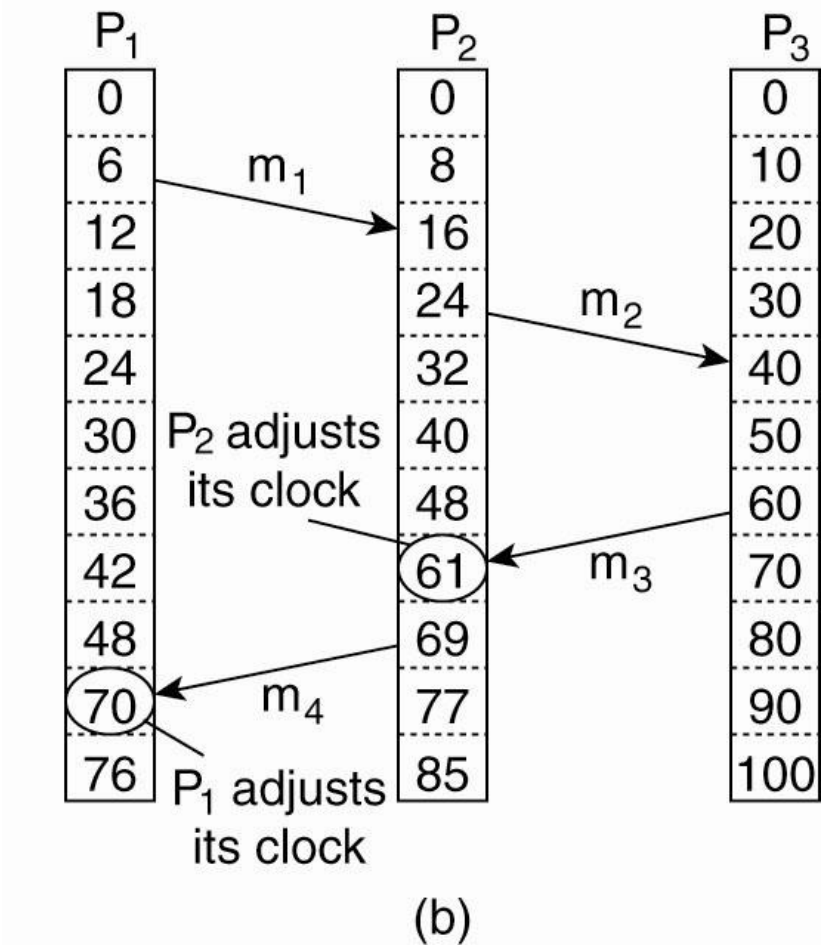


Figure 6-9. (b) Lamport's algorithm corrects the clocks.

Lamport's Logical Clocks (4)

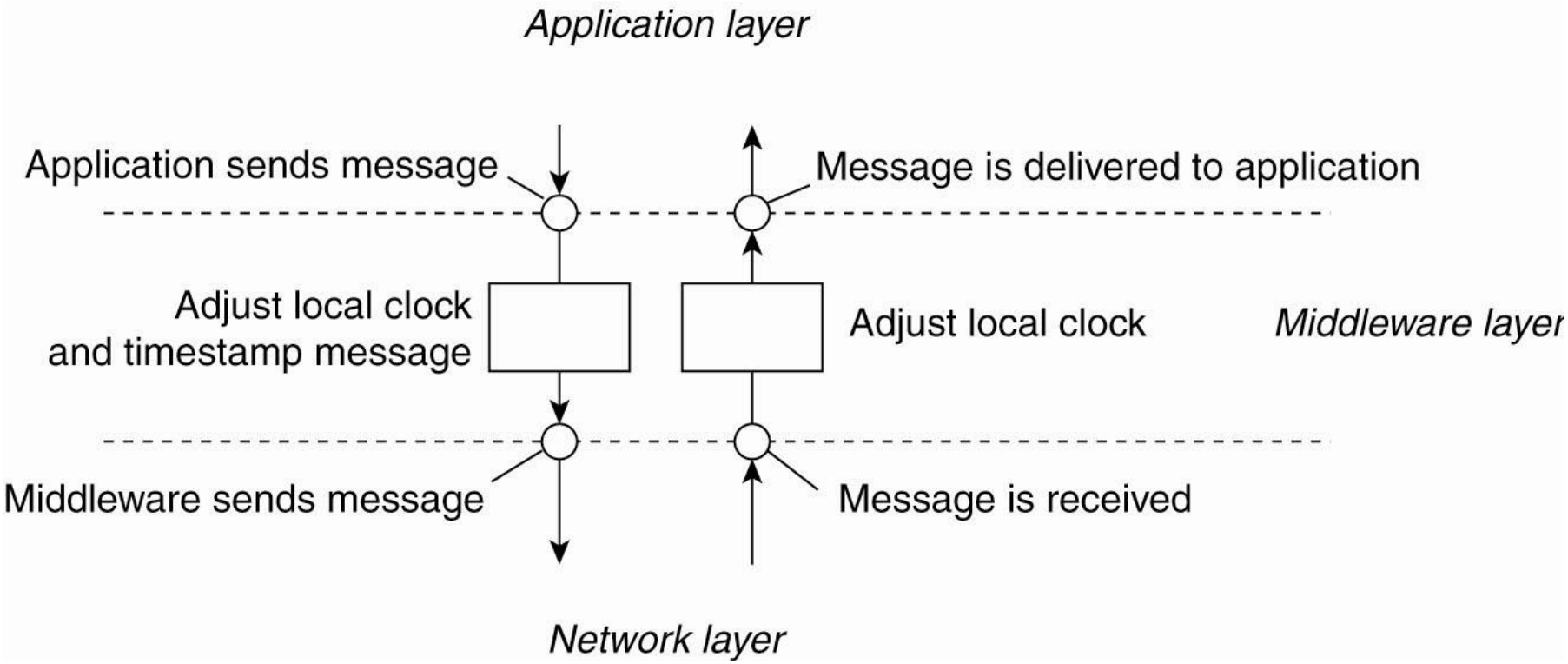


Figure 6-10. The positioning of Lamport's logical clocks in distributed systems.

Lamport's Logical Clocks (5)

Updating counter C_i for process P_i

1. Before executing an event P_i executes $C_i \leftarrow C_i + 1$.
2. When process P_i sends a message m to P_j, it sets *m*'s timestamp *ts (m)* equal to C_i after having executed the previous step.
3. Upon the receipt of a message *m*, process P_j adjusts its own local counter as $C_j \leftarrow \max\{C_j, ts(m)\}$, after which it then executes the first step and delivers the message to the application.

Partial order logical clocks assign timestamps to distributed events to create a causal, but not necessarily total, ordering, capturing the "happened-before" relationship without relying on physical clocks.

This establishes that one event could have caused another, but not all events can be compared (i.e., some events are concurrent).

Lamport's algorithm uses simple scalar timestamps for a partial ordering, while vector clocks provide a more complete ordering, enabling detection of concurrent events.

Example: Totally Ordered Multicasting

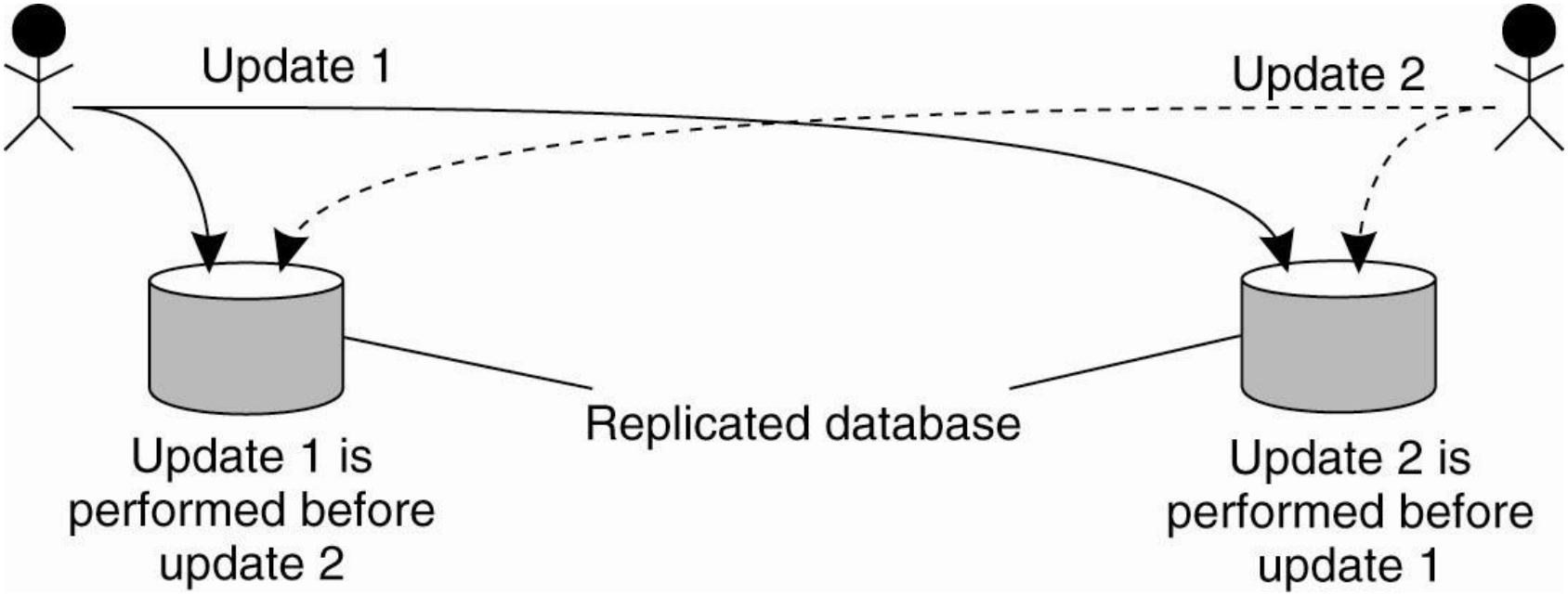


Figure 6-11. Updating a replicated database and leaving it in an inconsistent state.

Vector Clocks (1)

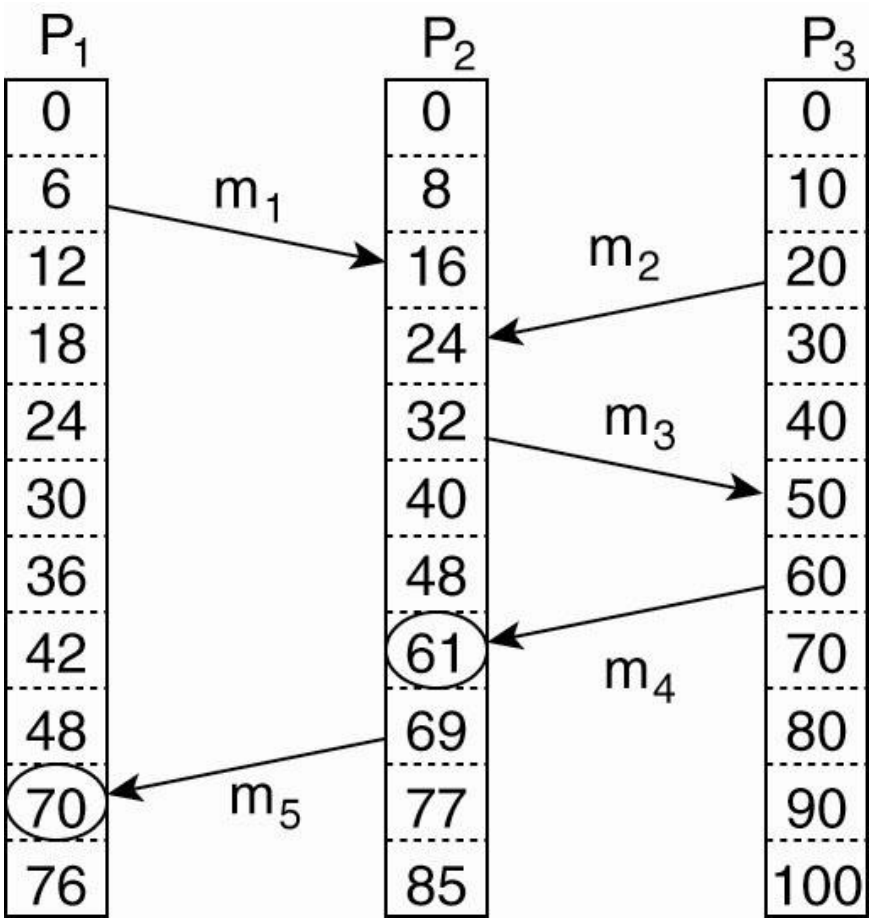


Figure 6-12. Concurrent message transmission using logical clocks.

Vector Clocks (2)

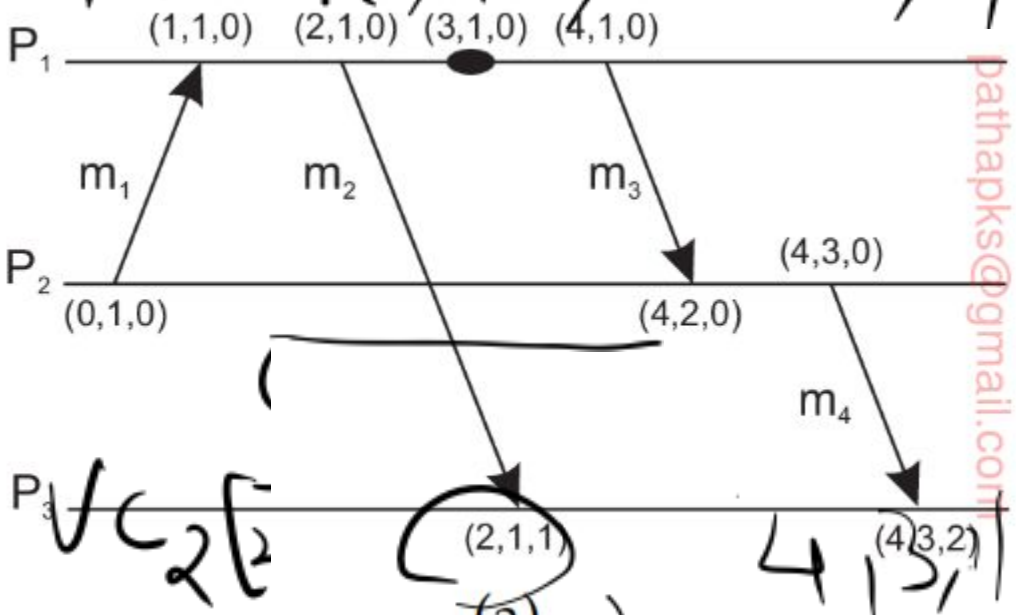
Vector clocks are constructed by letting each process P_i maintain a vector VC_i with the following two properties:

1. $VC_i [i]$ is the number of events that have occurred so far at P_i. In other words, $VC_i [i]$ is the local logical clock at process P_i .
2. If $VC_i [j] = k$ then P_i knows that k events have occurred at P_j. It is thus P_i's knowledge of the local time at P_j .

Vector Clocks (3)

Steps carried out to accomplish property 2 of previous slide:

1. Before executing an event P_i executes $VC_i[i] \leftarrow VC_i[i] + 1$.
2. When process P_i sends a message m to P_j, it sets *m*'s (vector) timestamp *ts (m)* equal to VC_i after having executed the previous step.
3. Upon the receipt of a message m, process P_j adjusts its own vector by setting $VC_j[k] \leftarrow \max\{VC_j[k], ts(m)[k]\}$ for each *k*, after which it executes the first step and delivers the message to the application.



(a)

pathapks@gmail.com

Physical clocks

Problem

Sometimes we simply need the exact time, not just an ordering.

Solution: Universal Coordinated Time (UTC)

- Based on the number of transitions per second of the cesium 133 atom (pretty accurate).
- At present, the real time is taken as the average of some 50 cesium clocks around the world.
- Introduces a leap second from time to time to compensate that days are getting longer.

Note

UTC is broadcast through short-wave radio and satellite. Satellites can give an accuracy of about ± 0.5 ms.

Cristian's (Time Server) Algorithm

Uses a *time server* to synchronize clocks

- Time server keeps the reference time (say UTC)
- A client asks the time server for time, the server responds with its current time, and the client uses the received value *T* to set its clock

But network round-trip time introduces errors…

- Let **RTT = response-received-time – request-sent-time** (measurable at client),
- If we know (a) min = minimum client-server one-way transmission time and (b) that the server timestamped the message at the last possible instant before sending it back
- Then, the actual time could be between **[T+min,T+RTT— min]**

Consider a time server A. Another server B requests time from A at 10:37:640 (hr, min, 1/100 sec.), At 10:37:710, server B receives a reply from timeserver A with the timestamp of 10:37:375.

Find out the drift of B's clock with respect to the time-server A's clock (assume there is no processing time at the time-server for time service).

ANS:

RTT: Reply – Request = 610-540 = 70 1/100sec.

Adjusted local time: Server + RTT/2 = 375 + 35 = 410 1/100sec.

Drift: Adjusted local time – local time = 410 – 710 = -300 1/100sec. = -3 sec

Network Time Protocol (NTP)

Most widely used physical clock synchronization protocol on the Internet

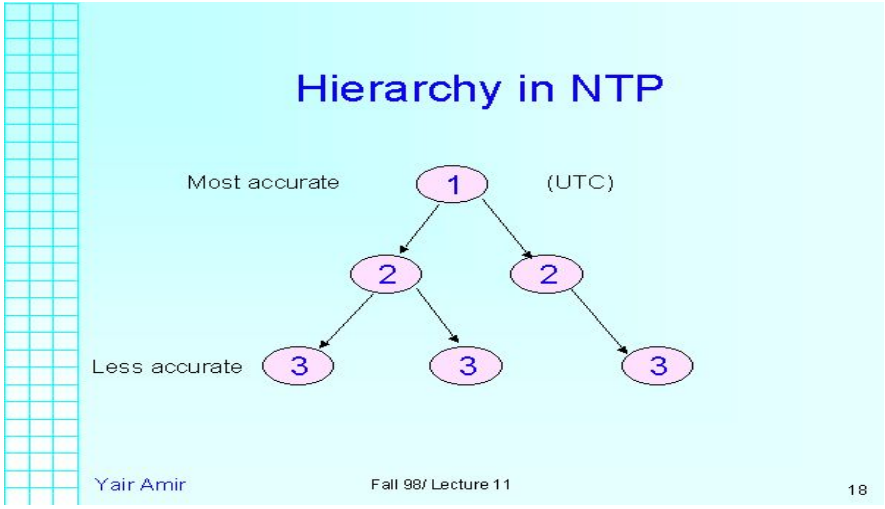
10-20 million NTP servers and clients in the Internet

Claimed Accuracy (Varies)

- milliseconds on WANs, submilliseconds on LANs

Hierarchical tree of time servers.

- The primary server at the root synchronizes with the UTC.
- Secondary servers - backup to primary server.
- Lowest
  - synchronization subnet with clients.



Berkeley UNIX algorithm

One daemon without UTC

Periodically, this daemon polls and asks all the machines for their time

The machines respond.

The daemon computes an average time and then broadcasts this average time.

The Berkeley Algorithm (1)

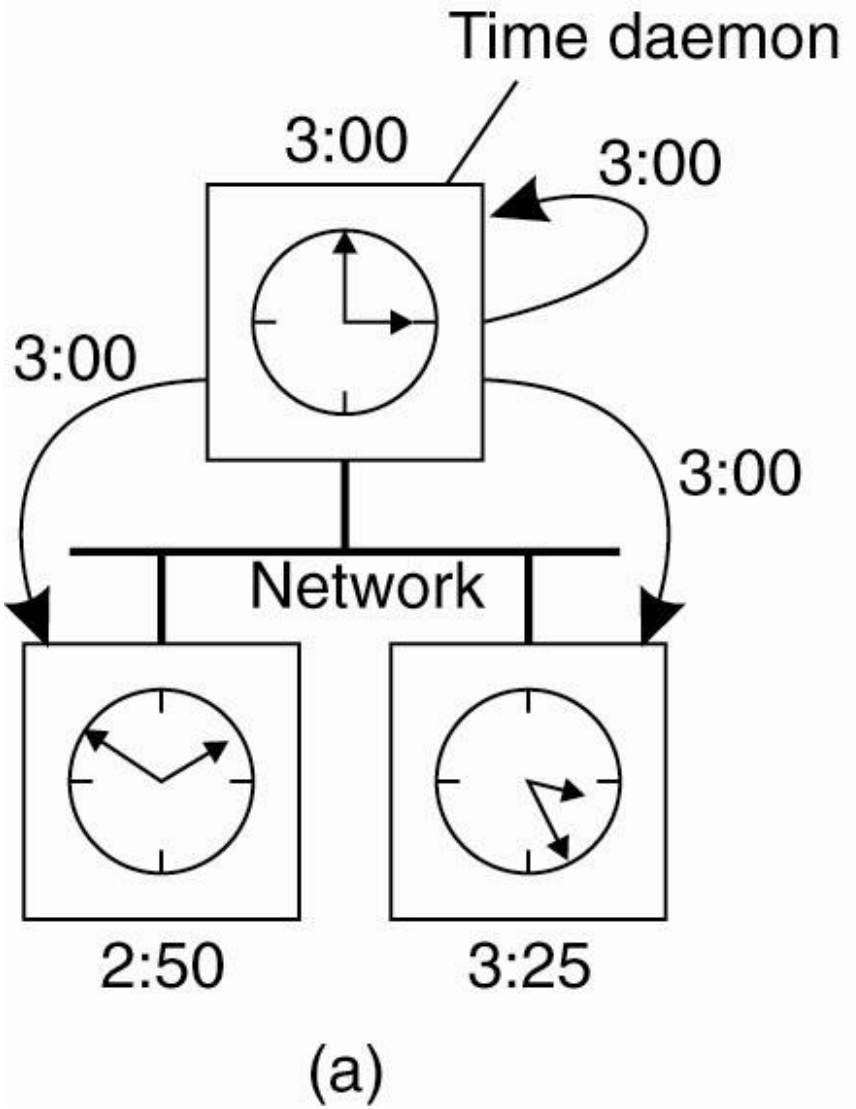


Figure 6-7. (a) The time daemon asks all the other machines for their clock values.

The Berkeley Algorithm (2)

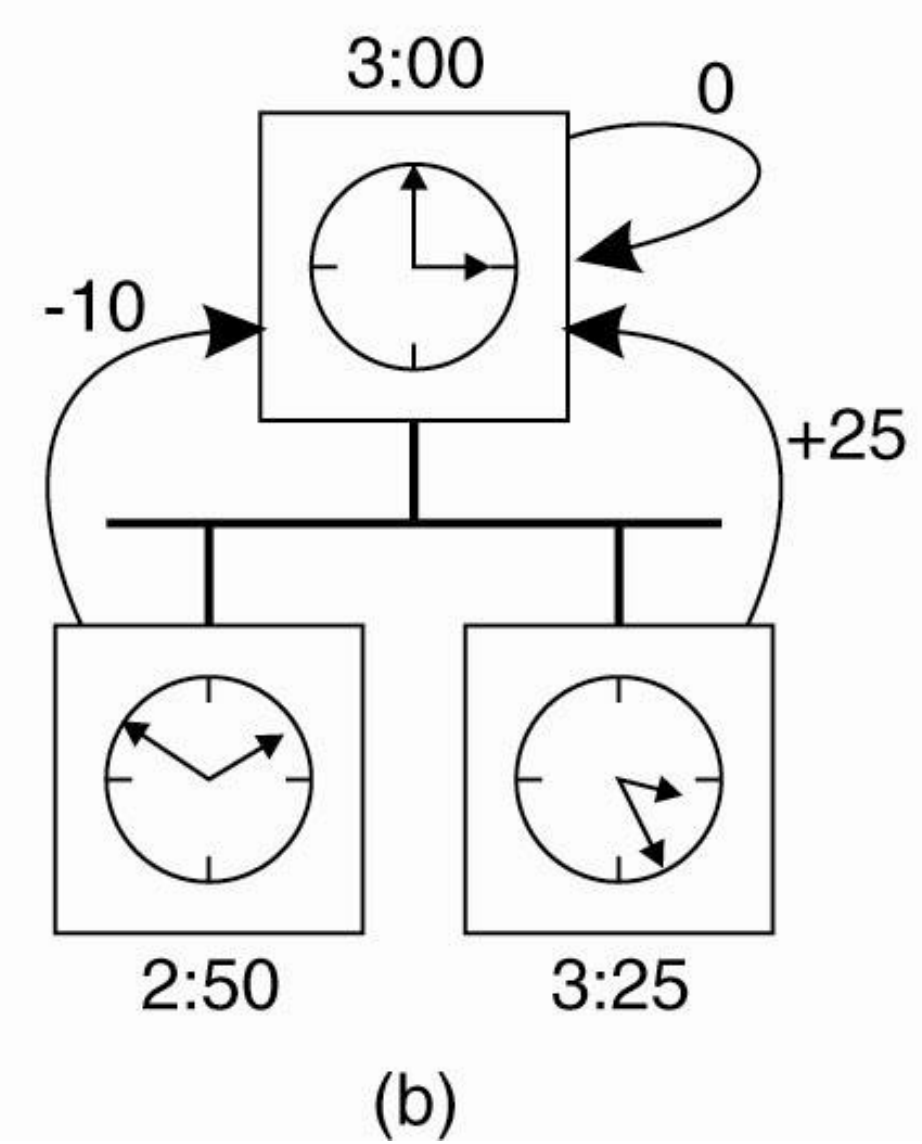


Figure 6-7.

(b) The machines answer.